

Especificação de Projeto: Caches Inteligentes em RISC-V (FPGA)

Disciplina: Projeto Integrador IV

Semestre: 2026/1

Professor: Bruno S. Neves

Objetivo: Implementação e Avaliação de Algoritmos de Substituição de blocos em Cache baseados em IA e Heurísticas Adaptativas.

Organização das equipes: duplas ou trios (máximo, irrevogável).

1. Introdução e Objetivo

O objetivo deste trabalho é projetar, implementar, validar e avaliar um algoritmo para substituição de bloco em cache, construído com base em técnicas de **Aprendizado de Máquina (ML) ou Heurísticas de Predição**, e aplicado a um processador RISC-V, equipado com duas caches L1, uma para dados e outra para código, e uma cache L2 exclusiva para dados, a ser unificada para comportar dados e instruções). O desafio consiste em superar o desempenho do algoritmo padrão LRU (*Least Recently Used*), respeitando as restrições de área e temporização (MHz) de um FPGA alvo (Ciclone III EP3C25F324C6). Diferentes arranjos de cache em termos de parâmetros como capacidade de dados, tamanho de bloco e grau de associatividade devem ser testados em combinação com os dois algoritmos, LRU e AIRA (*AI based Replacement Algorithm*), buscando expor limites de ganhos alcançáveis.

Link para download do processador: a ser disponibilizado em breve.

2. Escopo Técnico

- **Processador Base:** RISC-V (RV32I) em ambiente *bare-metal* (sem SO).
- **Hardware:** Implementação em HDL (Verilog) para FPGA.
- **Ferramentas:** Simuladores (como o Gem5 e o QtRVSim - <https://comparch.edu.cvut.cz/qtrvsim/app/>) e ferramentas de projeto RTL (Quartus e Modesim).

3. Possíveis algoritmos para desenvolvimento (Distribuição por Equipe)

Abaixo estão elencados 8 algoritmos de substituição que utilizam técnicas de Aprendizado de Máquina (ML) ou Heurísticas de Predição, variando em complexidade e área. Estes algoritmos serão distribuídos entre as equipes, de modo que cada algoritmo seja abordado pelo mesmo quantitativo de equipes (máximo de duas equipes para um mesmo tema/algoritmo). Outros algoritmos foram desta lista também podem ser desenvolvidos, sobretudo em uma perspectiva de interesse das equipes em desenvolver um estudo com algum algoritmo mais do seu interesse. Os algoritmos listados não esgotam, portanto, o rol de soluções baseadas em Aprendizado de Máquina (ML) ou Heurísticas de Predição, havendo outras opções potencialmente ainda mais eficientes e desafiadoras. Cada equipe deve enviar um e-mail para brunoneves@unipampa.edu.br até **23:59 de 21/03/2026** informando: (1) nomes dos seus integrantes e (2) tema/algoritmo escolhido para desenvolvimento e preencher com estes mesmos dados a tabela disponível em:

https://docs.google.com/spreadsheets/d/1mR_uepelxm15tcrJdv6N2lgsp0_dDL4FkNn1S6K1Kbl/edit?usp=sharing

Sugere-se realizar uma consulta a tabela antes de enviar o e-mail para evitar repetição de temas (máximo de duas equipes para um mesmo tema/algoritmo). Após o prazo informado, o professor revisará se a distribuição dos temas está equilibrada e aplicará ajustes na distribuição dos temas, caso necessário. Obs.: algoritmos mais complexos permitem uma exploração mais rica do espaço de projeto e, conseqüente, a escolha por estes algoritmos possibilita mais facilmente que inovações científicas ou tecnológicas sejam alcançadas durante o desenvolvimento deste trabalho, podendo resultar em um convite aos respectivos discentes para publicação dos resultados.

Algoritmos:

1. **DRRIP:** Predição de intervalo de re-referência bimodal.
2. **Leaky Bucket:** Frequência de acesso com decaimento temporal.
3. **SHIP:** Assinaturas de instrução (PC) para predição de reuso.
4. **Hawkeye:** Mimetismo do Ótimo de Belady via histórico de PC.
5. **Perceptron:** Preditor baseado em pesos e produto escalar.
6. **Mockingjay:** Estimativa de intervalo de tempo até o próximo reuso.
7. **Linear Regression:** Decisão multivariável baseada em funções lineares.
8. **Q-Learning (Tabelado):** Aprendizado por reforço para seleção de política de inserção.

4. Configuração da Hierarquia de Memória e Espaço de Parâmetros

Para que os ganhos da substituição inteligente sejam observáveis, as caches devem ser configuradas para operar em cenários de **pressão de memória**. Assim, os testes para avaliação de desempenho do algoritmo de cacheamento (a ser aplicado em todas as caches) devem ser realizados com os seguintes parâmetros:

Parâmetro	Cache L1 (Dados)	Cache L2 (Unificada)
Capacidade de armazenamento	4 KB a 8 KB	32 KB a 128 KB
Tamanho do Bloco	32 Bytes	64 Bytes
Associatividade	2-vias ou 4-vias	8-vias ou 16-vias
Política de Escrita	A atual implementada	A atual implementada

5. Cronograma de Desenvolvimento (Sprints Semanais)

- **Semanas 1-4 (Modelagem):** Estudo teórico, modelagem em Python/C e validação do *Hit Rate* lógico contra o baseline LRU.
- **Semanas 5-8 (HDL):** Descrição das tabelas de pesos/contadores e lógica de decisão (inferência) e atualização (treinamento).
- **Semanas 9-12 (Integração):** Inserção do AIRA no core RISC-V, depuração em hardware (Modelsim e SignalTap) e execução de benchmarks que desafiem o algoritmo, tais como CoreMark, Matrizes e Grafos, etc. Os benchmarks para teste comparativo dos algoritmos estão disponíveis no Apêndice A desta especificação e online no seguinte link: <https://drive.google.com/file/d/1NjjGcwhRblQyx-GqRt7r7IOjLF4l7uWM/view?usp=sharing>. As equipes complementarmente podem levantar outros os benchmarks relevantes/necessários para teste dos dois algoritmos de substituição (LRU e AIRA).

6. Análise de Overhead e Eficiência

As equipes devem quantificar o "custo" da inteligência adicionada através das seguintes métricas:

- **Área:** Número de Elementos Lógicos (*Logical Elements - LEs*), Registradores e BRAMs do FPGA adicionais em relação ao RV32I básico.
- **Timing:** Impacto na frequência máxima (F_{max}) do processador.
- **Latência:** Ciclos de clock necessários para a tomada de decisão da IA.
- **Índice de Mérito (IEC):** Ganho percentual de *Hit Rate* dividido pela quantidade de LEs extras.

7. Estrutura do Repositório (Padrão Git)

A entrega deve seguir rigorosamente esta estrutura de pastas:

- /docs: Relatório final (PDF) e diagramas.
- /rtl: Código fonte (subpastas cache/ e riscv_core/).
- /sim: Testbenches e scripts de simulação.
- /software: Código-fonte dos benchmarks em C/Assembly.
- /synth: Relatórios de síntese (utilização e timing).
- README.md: Instruções para uso de cada projeto e tabela resumo de resultados.

8. Formato do Relatório Final

O relatório deve seguir o formato de artigo científico (máx. 10 páginas) contendo:

1. **Arquitetura Proposta:** Detalhamento da lógica de IA em hardware.
2. **Metodologia:** passos de desenvolvimento do trabalho.
3. **Resultados de síntese e desempenho:** Gráficos comparativos de *taxa de acertos*, *área*, *frequência* e *potência* para diferentes organizações de cache (capacidade, associatividade, etc), buscando evidenciar o impacto da organização no custo e desempenho do algoritmo de substituição. Em análises de desempenho, avaliar com todos os benchmarks selecionados.

4. **Análise Crítica:** Discussão sobre o custo-benefício da solução.

Observação: também é esperada a comparação de resultados entre os trabalhos. Para tal, serão utilizadas tabelas comparativas como a Tabela de Referência no Apêndice B desta especificação.

9. **Gestão de Projeto e Colaboração (Requisito Obrigatório)**

Para garantir a organização e o cumprimento desta especificação, cada equipe deverá utilizar o **Trello** (ou ferramenta similar de Kanban) para gerenciar o desenvolvimento do projeto. A gestão visual permitirá que tanto a equipe quanto o professor acompanhem o progresso das Sprints em tempo real.

9.1. **Estrutura do Quadro Sugerida**

O quadro deve ser dividido minimamente nas seguintes colunas:

- **Backlog:** Todas as tarefas planejadas até o fim do semestre (Modelagem IA, Descrição HDL, Unificação L2, Benchmarking, etc.).
- **Sprint Atual (To Do):** Tarefas selecionadas para a semana vigente.
- **Em Execução (Doing):** O que está sendo codificado ou simulado no momento.
- **Bloqueado (Blocked):** Tarefas impedidas por bugs de hardware ou falta de ferramentas.
- **Validado (Done):** Tarefas que passaram pela simulação RTL ou teste em FPGA e foram documentadas.

9.2. **Orientações para o Uso Eficaz:**

1. **Atomicidade:** Não criem cartões genéricos como "Fazer a Cache". Dividam em tarefas menores, como "Implementar Tabela de Pesos do Perceptron" ou "desenvolver lógica de inferência".
2. **Atribuição:** Cada cartão deve ter um responsável claro, evitando que partes do hardware fiquem "sem dono".
3. **Critérios de Aceite:** No verso de cada cartão, descrevam o que define a conclusão da tarefa (ex: "Passar no Testbench X com zero erros").

9.3. **Entrega da Gestão:**

O link para o quadro do Trello (em modo de visualização) deve ser enviado ao professor na **Semana 3**. A movimentação constante do quadro será um dos critérios de avaliação da maturidade técnica da equipe.

Apêndice A

(Benchmarks para avaliação comparativa dos algoritmos)

```
#include <stdio.h>
#include <stdlib.h>
#include <stdint.h>

// --- CONFIGURAÇÕES DE TAMANHO ---
// Ajuste para ser ~2x o tamanho da sua Cache L2 para forçar o Miss
#define L2_SIZE_BYTES (128 * 1024)
#define ARRAY_SIZE (L2_SIZE_BYTES * 2 / sizeof(int))

// --- MACROS PARA MÉTRICAS (PROFILING) ---
// No PC, você pode usar 'perf' externo. No RISC-V, insira leitura de CSRs aqui.
#define START_PERF() printf("\n[Iniciando medição...]\n")
#define END_PERF() printf("[Fim da medição.]\n\n")

typedef struct Node {
    int data;
    struct Node *next;
} Node;

// --- DEFINIÇÃO DOS BENCHMARKS ---

void run_streaming(int *array, volatile int *hot_data) {
    printf("Executando: Streaming + HotSet (Antagonista ao LRU)\n");
    START_PERF();
    for (int it = 0; it < 10; it++) {
        for (int i = 0; i < ARRAY_SIZE; i++) {
            array[i] += i;
            if (i % 64 == 0) *hot_data += array[i];
        }
    }
    END_PERF();
}

void run_matrix_conv(int *img, int *out) {
    printf("Executando: Matriz 2D - Convolução (Reuso em Janela)\n");
    int width = 128;
    int height = ARRAY_SIZE / width;
    START_PERF();
    for (int y = 1; y < height - 1; y++) {
        for (int x = 1; x < width - 1; x++) {
            out[y*width+x] = img[(y-1)*width+x] + img[y*width+x] + img[(y+1)*width+x];
        }
    }
    END_PERF();
}

void run_linked_list(Node *nodes, int count) {
```

```

printf("Executando: Linked List (Ponteiros/Saltos de Memória)\n");
Node *curr = nodes;
START_PERF();
for (int i = 0; i < count * 50; i++) {
    curr->data += i;
    curr = curr->next;
}
END_PERF();
}

```

```

void run_pattern_search(uint8_t *blob, int size) {
    printf("Executando: Pattern Search (Estresse de L2 Unificada)\n");
    START_PERF();
    for (int i = 1024; i < size; i++) {
        for (int j = 1; j < 64; j++) {
            if (blob[i] == blob[i-j]) { blob[i]++; break; }
        }
    }
    END_PERF();
}

```

```

// --- MENU PRINCIPAL ---

```

```

void print_menu() {
    printf("=====\n");
    printf(" SELETOR DE BENCHMARKS - CACHE IA  \n");
    printf("=====\n");
    printf("1. Streaming + HotSet (L1/L2 Data)\n");
    printf("2. Matrix Convolution (Temporal Reuse)\n");
    printf("3. Linked List Traversal (Pointer Chasing)\n");
    printf("4. Pattern Search (L2 Unified Stress)\n");
    printf("5. Executar Todos em Sequência\n");
    printf("0. Sair\n");
    printf("Escolha uma opção: ");
}

```

```

int main() {
    int choice = -1;
    volatile int hot_val = 0;

    // Alocação e Inicialização
    int *big_array = (int *)calloc(ARRAY_SIZE, sizeof(int));
    int *out_array = (int *)calloc(ARRAY_SIZE, sizeof(int));
    uint8_t *blob = (uint8_t *)malloc(L2_SIZE_BYTES);

    Node *nodes = (Node *)malloc(2000 * sizeof(Node));
    for(int i=0; i<1999; i++) nodes[i].next = &nodes[i+1];
    nodes[1999].next = &nodes[0];

    while (choice != 0) {
        print_menu();
        if (scanf("%d", &choice) != 1) break;
    }
}

```

```
switch (choice) {
    case 1: run_streaming(big_array, &hot_val); break;
    case 2: run_matrix_conv(big_array, out_array); break;
    case 3: run_linked_list(nodes, 2000); break;
    case 4: run_pattern_search(blob, L2_SIZE_BYTES); break;
    case 5:
        run_streaming(big_array, &hot_val);
        run_matrix_conv(big_array, out_array);
        run_linked_list(nodes, 2000);
        run_pattern_search(blob, L2_SIZE_BYTES);
        break;
    case 0: printf("Encerrando...\n"); break;
    default: printf("Opção inválida!\n");
}
}

free(big_array); free(out_array); free(nodes); free(blob);
return 0;
}
```

Apêndice B

(Tabela de Referência para os Alunos - Métricas de Entrega)

BENCHMARK A					
(L1 4KB e 2 vias, L2 32KB e 8 vias)					
Métrica	Baseline (LRU)	AIRA1	Impacto	AIRA2	Impacto
Taxa de Acerto (%)	%	%	+ %	%	+ %
Área (LEs)	un.	un.	+ %	un.	+ %
Frequência (Fmax)	MHz	MHz	- %	MHz	- %
Latência de Decisão	1 ciclo	Ciclos	+ ciclos	Ciclos	+ ciclos

BENCHMARK B					
(L1 4KB e 2 vias, L2 32KB e 8 vias)					
Métrica	Baseline (LRU)	AIRA1	Impacto	AIRA2	Impacto
Taxa de Acerto (%)	%	%	+ %	%	+ %
Área (LEs)	un.	un.	+ %	un.	+ %
Frequência (Fmax)	MHz	MHz	- %	MHz	- %
Latência de Decisão	1 ciclo	Ciclos	+ ciclos	Ciclos	+ ciclos